Project File

Web Programming using

Python and JavaScript

Name: Vansh Sharma

Course: B-Tech CSE Core

Year: 2nd Year, 4th Semester

Roll No: 2401010045

Submitted To:

Ms. Mansi Kajal

EXPERIMENT – I

❖ **Aim :**

Creating a **Personal Portfolio Web Application** using **HTML, CSS, and JavaScript**.

❖ **Tasks :**

- **Project Setup & Introduction** – Create project folder and basic linked files.
- **HTML Structure** – Build portfolio sections using semantic HTML.
- **Styling with CSS** – Apply layout, colors, spacing, and neat design.
- **Projects & Skills Display** – Show skills and projects clearly on the page.
- **JavaScript Interactivity** – Add basic interactive functionality using JavaScript.
- **Contact Form (Bonus)** – Create a form with user submission feedback.

❖ **Tech Stack Used :**

- **HTML** , **CSS** and **JavaScript**

❖ Code :

❯ **HTML CODE :**

```
<!--  -->
<!-- Name: Vansh
Project: Personal Portfolio -->
<!--  -->
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title> Vansh Sharma Portfolio</title>
   <link rel="stylesheet" href="style.css">
```

```
</head>

<body>

<header>
  <h1> Vansh </h1>
  <p>I am learning Web Development</p>
</header>

<nav>
  <a href="#skills">Skills</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>

<section class="welcome">
  <h2>Welcome</h2>
  <p>This is my beginner portfolio website.</p>
  <button onclick="showMessage()">Click Me</button>
</section>

<section id="skills">
  <h2>
    My Skills
    <button onclick="toggleSection('skillsContent')">Hide / Show</button>
  </h2>
  <ul id="skillsContent">
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript (Beginner)</li>
  </ul>
```

```
</section>

<section id="projects">
  <h2>
    My Projects
    <button onclick="toggleSection('projectsContent')">Hide / Show</button>
  </h2>

  <div id="projectsContent">
    <div class="card">
      <h3>Portfolio Website</h3>
      <p>This is my personal portfolio made using HTML, CSS and JavaScript.</p>
    </div>

    <div class="card">
      <h3>Simple HTML Page</h3>
      <p>A basic web page created using only HTML.</p>
    </div>
  </div>
</section>

<section id="contact">
  <h2>Contact Me</h2>

  <form onsubmit="submitForm(); return false;">
    <label>Name:</label>
    <input type="text" required>

    <label>Email:</label>
    <input type="email" required>
```

```
        <label>Message:</label>
        <textarea rows="4" required></textarea>

        <button type="submit">Submit</button>
    </form>

    <p id="result"></p>

    <a href=" https://github.com/VanshSharma27?tab=repositories " target="_blank">GitHub</a> |
    <a href=" www.linkedin.com/in/vansh-sharma-3b6a40388 " target="_blank">LinkedIn</a>
</section>

<footer>
    <p>© 2026 Vansh </p>
</footer>

<button id="scrollBtn" onclick="scrollToTop()">⬆  </button>

<script src="script.js"></script>
</body>
</html>
```

> CSS CODE :

```
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&display=swap');
* {
    box-sizing: border-box;
}
body {
    margin: 0;
    font-family: 'Poppins', sans-serif;
    background: linear-gradient(to right, #fbc2eb, #a6c1ee);
```

```
}
header {
   background: purple ;
   color: white;
   padding: 40px 20px;
   text-align: center;
}
header h1 {
   margin: 0;
   font-size: 38px;
}
header p {
   font-size: 18px;
   opacity: 0.9;
}
nav {
   background-color: #ffffff;
   padding: 12px;
   text-align: center;
   box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}
nav a {
   color: #0a89dd;
   text-decoration: none;
   margin: 0 15px;
   font-weight: 600;
}
nav a:hover {
   color: blueviolet;
}
```

```
section {
    background-color: white;
    max-width: 900px;
    margin: 30px auto;
    padding: 25px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.15);
}
.welcome {
    text-align: center;
}
.welcome h2 {
    font-size: 28px;
}
button {
    background: linear-gradient(to right, #6a11cb, #2575fc);
    color: white;
    border: none;
    padding: 10px 18px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 14px;
    transition: 0.3s;
}
button:hover {
    transform: scale(1.05);
}
.card {
    background: #f9f9f9;
    border-radius: 10px;
    padding: 20px;
```

```
    margin: 15px 0;
    transition: 0.3s;
}
.card:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}
ul {
    list-style: none;
    padding: 0;
}
ul li {
    background: #eef3ff;
    margin: 8px 0;
    padding: 10px;
    border-radius: 6px;
    font-weight: 500;
}
form {
    margin-top: 15px;
}
input, textarea {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    border-radius: 8px;
    border: 1px solid #ccc;
    font-family: 'Poppins', sans-serif;
}
input:focus, textarea:focus {
    outline: none;
```

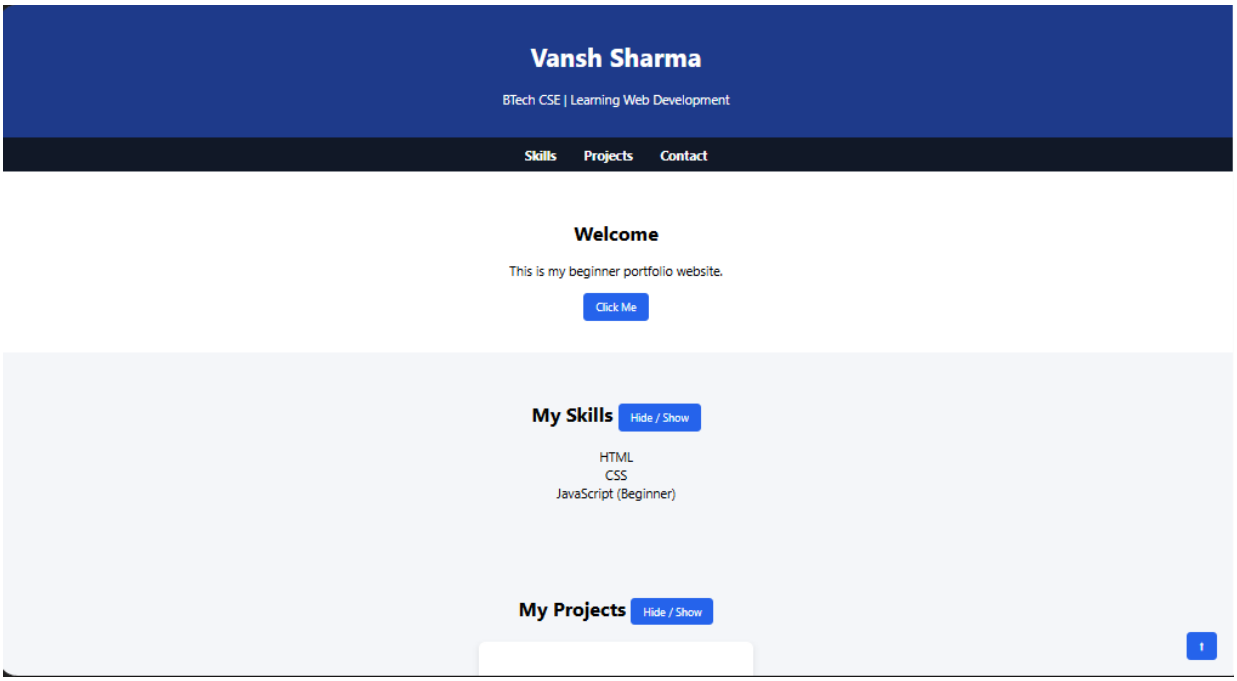
```
    border-color: #2575fc;
}
#contact a {
    display: inline-block;
    margin: 10px 10px 0 0;
    text-decoration: none;
    font-weight: 600;
    color: #2575fc;
}
#contact a:hover {
    color: #6a11cb;
}
footer {
    background-color: #111;
    color: white;
    text-align: center;
    padding: 15px;
    margin-top: 40px;
}
#scrollBtn {
    position: fixed;
    bottom: 25px;
    right: 25px;
    display: none;
    background: linear-gradient(to right, #6a11cb, #2575fc);
    color: white;
    border-radius: 50%;
    border: none;
    padding: 12px 16px;
    font-size: 18px;
    box-shadow: 0 8px 15px rgba(0,0,0,0.3);
```

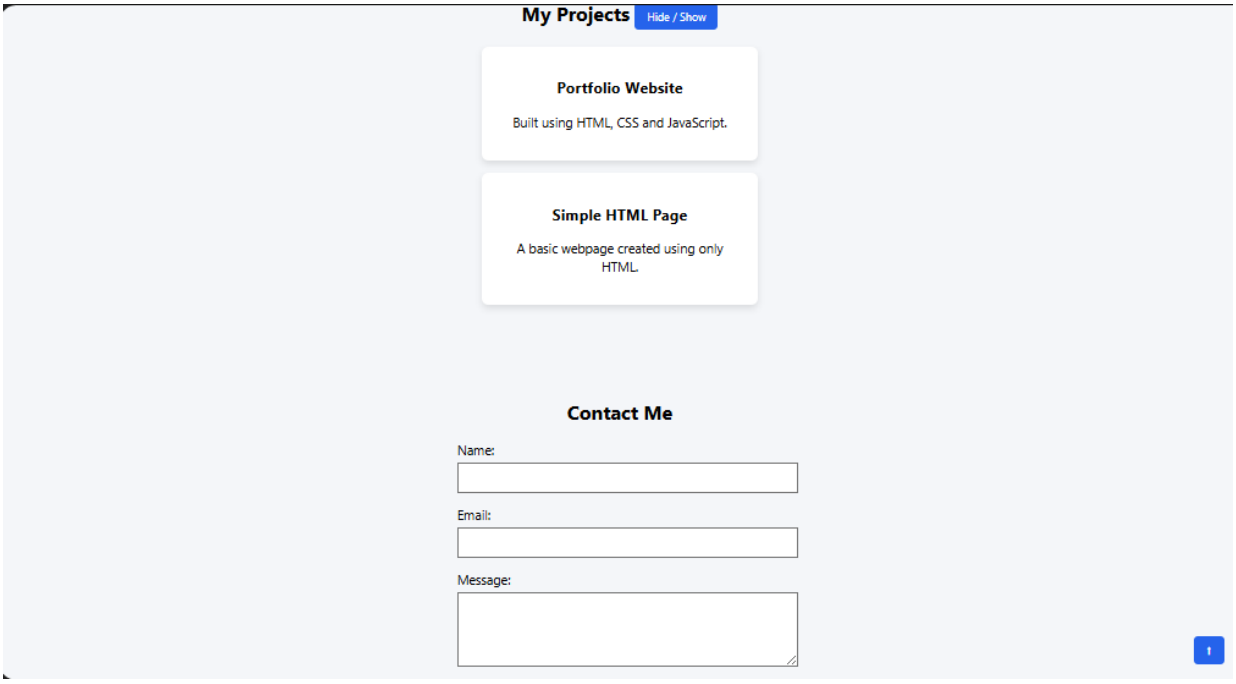
}

> JS **CODE :**

```
function showMessage() {
   alert("Thank you for visiting");
}
function showMessage() {
   alert("Welcome to my portfolio website!");
}
function submitForm() {
   document.getElementById("result").innerText =
      "Thank you! Your message has been submitted.";
}
function toggleSection(id) {
   const section = document.getElementById(id);
   if (section.style.display === "none") {
      section.style.display = "block";
   } else {
      section.style.display = "none";
   }
}
window.onscroll = function () {
   const btn = document.getElementById("scrollBtn");
   if (document.documentElement.scrollTop > 200) {
      btn.style.display = "block";
   } else {
      btn.style.display = "none";
   }
};
function scrollToTop() {
   window.scrollTo({ top: 0, behavior: "smooth" });
}
```

❖ **OUTPUT:**


Vansh Sharma
BTech CSE | Learning Web Development
Skills
Projects
Contact
Welcome
This is my beginner portfolio website.
Click Me
My Skills
Hide / Show
HTML
CSS
JavaScript (Beginner)
My Projects
Hide / Show


My Projects
Hide / Show
Portfolio Website
Built using HTML, CSS and JavaScript.
Simple HTML Page
A basic webpage created using only HTML.
Contact Me
Name:
Email:
Message:

EXPERIMENT – II

❖ **Aim :**

Creating a **Company Landing Page** using **HTML and CSS** to display company information, services, testimonials, and contact details.

❖ **Tasks :**

- **Project Setup & Introduction** – Create project folder and basic HTML and CSS files.
- **HTML Structure** – Build landing page sections using semantic HTML tags.
- **Styling with CSS** – Apply layout, colors, fonts, spacing, and borders for a professional design.
- **Services Section Design** – Display services with service name and short description.
- **Testimonials Section** – Add customer feedback and style it neatly.
- **Contact Section (Bonus)** – Create contact information and a simple contact form.

❖ **Tech Stack Used :**

- **HTML , CSS and Javascript**

❖ **CODES :**

〉HTML CODE :

```
<!--  -->

<!-- Name: Vansh

Project: Vansh Solutions -->

<!--  -->

<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <title>Company Landing Page</title>

   <link rel="stylesheet" href="style.css">

</head><wbr>


</wbr>
```

```
<body>
  <header>
    <h1> Vansh  Solutions</h1>
    <p>Innovating Technology for a Better Tomorrow</p>
  </header>
  <section>
    <h2>About Us</h2>
    <p>
      Vansh Solutions provides modern and reliable
      technology services for businesses and individuals.
    </p>
  </section>

  <section>
    <h2>Our Services</h2>

    <div class="service-card">
      <h3>Web Development</h3>
      <p>We create simple and responsive websites.</p>
    </div>
    <div class="service-card">
      <h3>App Development</h3>
      <p>We develop easy-to-use mobile applications.</p>
    </div>

    <div class="service-card">
      <h3>Technical Support</h3>
      <p>We provide 24/7 technical assistance.</p>
    </div>
  </section>
```

```
<section>
  <h2>Testimonials</h2>

  <div class="testimonial">
    <p>"Great service and friendly team!"</p>
    <h4>- Aayush</h4>
  </div>

  <div class="testimonial">
    <p>"Very professional and reliable company."</p>
    <h4>- Vansh</h4>
  </div>
</section>

<section>
  <h2>Contact Us</h2>

  <p>Email: Vansh@technova.com</p>
  <p>Phone: 9910500570</p>
  <p>Address: New Delhi, India</p>

  <form onsubmit="showMessage(); return false;">
    <input type="text" placeholder="Name" required><br><br>
    <input type="email" placeholder="Email" required><br><br>
    <textarea rows="4" placeholder="Message" required></textarea><br><br>
    <button type="submit">Send</button>
  </form>
</section>

<footer>
  <p>@ Vansh Solutions</p>
```

```
    </footer>

    <script src="script.js"></script>

</body>
</html>
```

> CSS CODE :

```
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&display=swap');
* {
    box-sizing: border-box;
    font-family: 'Poppins', sans-serif;
}
body {
    margin: 0;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: #333;
}
header {
    text-align: center;
    padding: 50px 20px;
    color: white;
}
header h1 {
    font-size: 42px;
    margin-bottom: 10px;
}
header p {
    font-size: 18px;
    opacity: 0.9;
}
```

```
section {
    background-color: white;
    max-width: 1000px;
    margin: 30px auto;
    padding: 30px;
    border-radius: 15px;
    box-shadow: 0 15px 30px rgba(0,0,0,0.2);
}
section h2 {
    text-align: center;
    margin-bottom: 20px;
    color: #667eea;
}
.service-card {
    background: #f4f6ff;
    border-radius: 12px;
    padding: 20px;
    margin: 15px 0;
    transition: 0.3s ease;
}
.service-card h3 {
    margin-top: 0;
    color: #333;
}
.service-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 10px 20px rgba(0,0,0,0.2);
}
.testimonial {
    background: #f9f9f9;
    border-left: 5px solid #667eea;
```

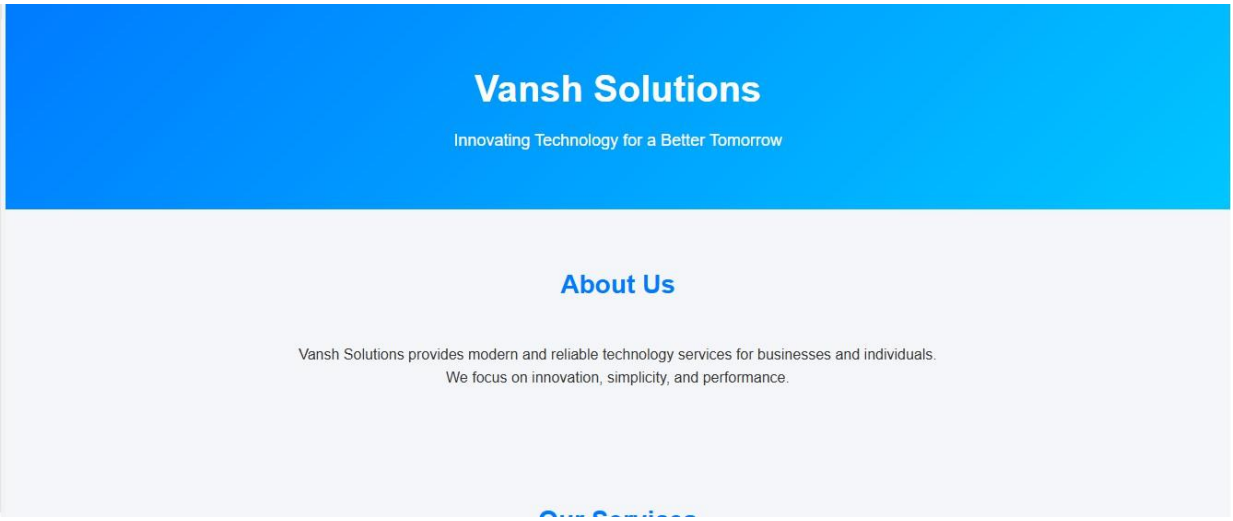
```
    padding: 15px 20px;
    margin: 15px 0;
    border-radius: 10px;
}
.testimonial p {
    font-style: italic;
}
.testimonial h4 {
    margin: 10px 0 0;
    text-align: right;
    color: #555;
}
section p {
    text-align: center;
}
form {
    max-width: 500px;
    margin: 20px auto 0;
    text-align: center;
}
input, textarea {
    width: 100%;
    padding: 12px;
    border-radius: 10px;
    border: 1px solid #ccc;
    font-size: 14px;
}
input:focus, textarea:focus {
    outline: none;
    border-color: #667eea;
}
```

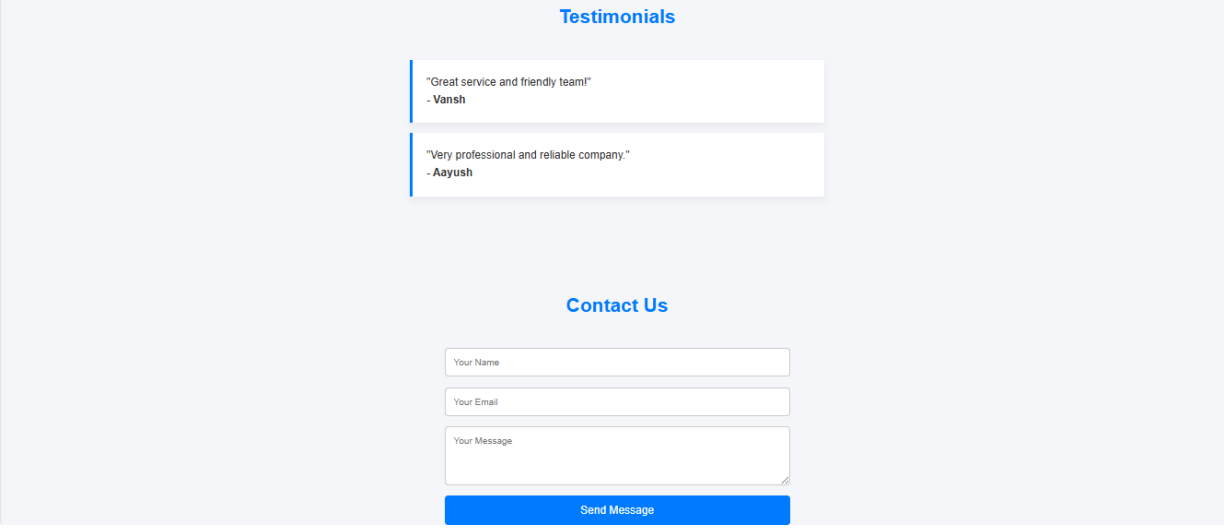
```
button {
    background: linear-gradient(to right, #667eea, #764ba2);
    color: white;
    border: none;
    padding: 12px 25px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 15px;
    transition: 0.3s ease;
}
button:hover {
    transform: scale(1.05);
    box-shadow: 0 8px 20px rgba(0,0,0,0.3);
}
footer {
    text-align: center;
    padding: 15px;
    color: white;
    opacity: 0.9;
}
@media (max-width: 768px) {
    header h1 {
        font-size: 32px;
    }
    section {
        margin: 20px;
        padding: 25px;
    }
}
```

> JS CODE :

```
function showMessage() {
   alert("Thank you for contacting Vansh Solutions!");
}
```

❖ OUTPUT :





EXPERIMENT – 3

❖ **Aim :**
To design and develop an **Interactive Quiz Web Application** using **HTML, CSS, and JavaScript**.

❖ **Tasks :**
• **Project Setup & Introduction** – Create the project folder and link HTML, CSS, and JavaScript files properly.
• **Quiz Interface Design** – Design quiz layout with questions, options, and navigation buttons using semantic HTML.
• **Styling with CSS** – Apply colors, spacing, alignment, and layout to improve user experience.
• **Quiz Logic Implementation** – Store questions and answers in JavaScript and handle user input.
• **Score Tracking & Navigation** – Move between questions and calculate the score dynamically.

❖ **Tech Stack Used :**
• **HTML** •**CSS** • **JavaScript**

❖ Html Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Interactive Quiz</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <h1>Interactive Quiz</h1>
    <p>Test your Web Development knowledge</p>
</header>
```

```html
<main class="quiz-container">

    <section id="intro">
        <h2>Quiz Introduction</h2>
        <p>This quiz contains multiple-choice questions. Click Start to begin.</p>
        <button onclick="startQuiz()">Start Quiz</button>
    </section>

    <section id="quiz" class="hidden">
        <h2 id="question"></h2>
        <div id="options"></div>
        <button onclick="nextQuestion()">Next</button>
    </section>

    <section id="result" class="hidden">
        <h2>Your Result</h2>
        <p id="scoreText"></p>
        <p id="feedback"></p>
        <button onclick="restartQuiz()">Restart Quiz</button>
    </section>

</main>
<script src="script.js"></script>
</body>
</html>
```

> CSS CODE :

```css
* {
    box-sizing: border-box;
```

```
    font-family: Arial, sans-serif;
}

body {
    margin: 0;
    background: linear-gradient(to right, #74ebd5, #9face6);
}

header {
    background-color:blueviolet ;
    color: white;
    padding: 20px;
    text-align: center;
}

.quiz-container {
    background-color: white;
    max-width: 600px;
    margin: 30px auto;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}

button {
    background-color: #3498db;
    color: white;
    border: none;
    padding: 10px 18px;
```

```
    margin-top: 15px;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background-color: #2980b9;
}

.option {
    background-color: #ecf0f1;
    padding: 10px;
    margin: 8px 0;
    border-radius: 5px;
    cursor: pointer;
}

.option:hover {
    background-color: #dcdde1;
}

.hidden {
    display: none;
}

footer {
    background-color: #2c3e50;
    color: white;
    text-align: center;
```

```
    padding: 10px;
}
```

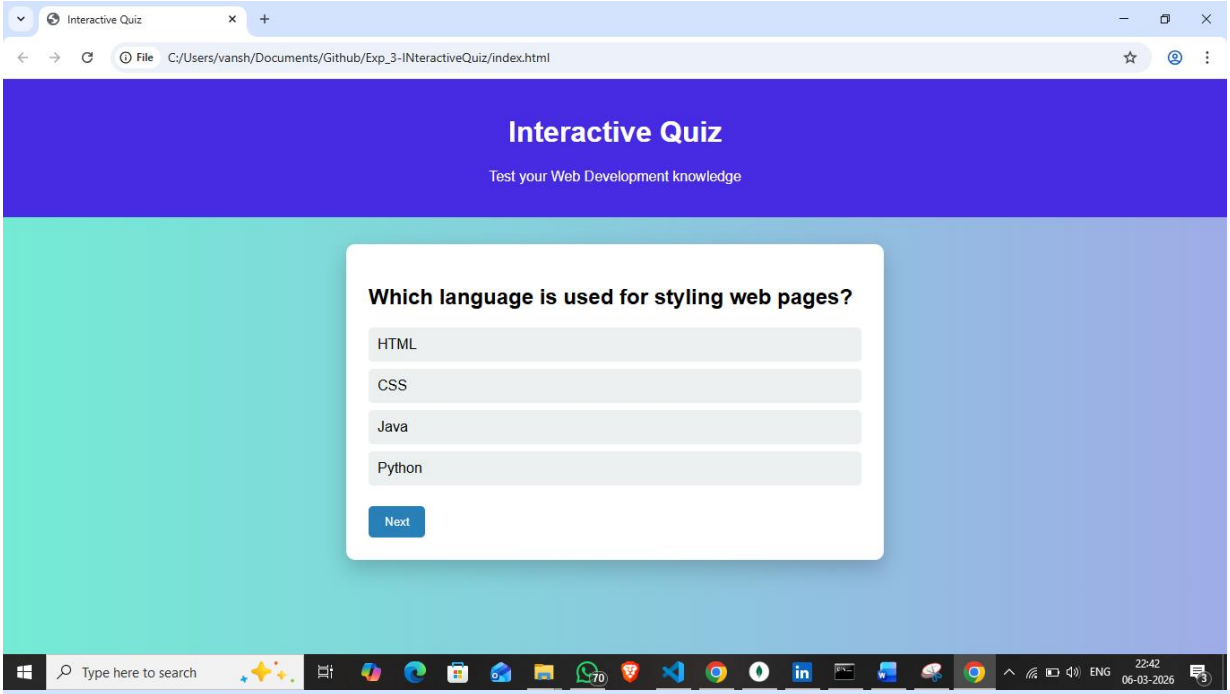
> JS **CODE :**

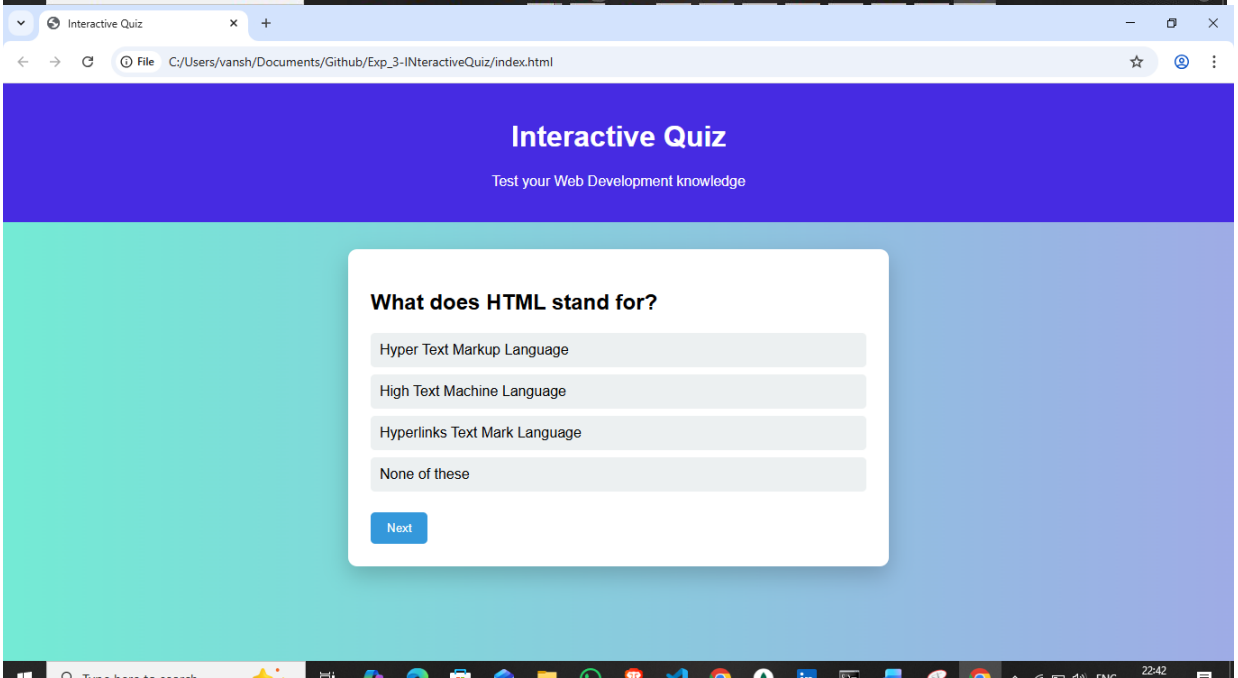
```
const quizData = [
    {
        question: "What does HTML stand for?",
        options: [
            "Hyper Text Markup Language",
            "High Text Machine Language",
            "Hyperlinks Text Mark Language",
            "None of these"
        ],
        correct: 0
    },
    {
        question: "Which language is used for styling web pages?",
        options: ["HTML", "CSS", "Java", "Python"],
        correct: 1
    },
    {
        question: "Which is used for web page interactivity?",
        options: ["HTML", "CSS", "JavaScript", "SQL"],
        correct: 2
    }
];

let currentQuestion = 0;
let score = 0;
```

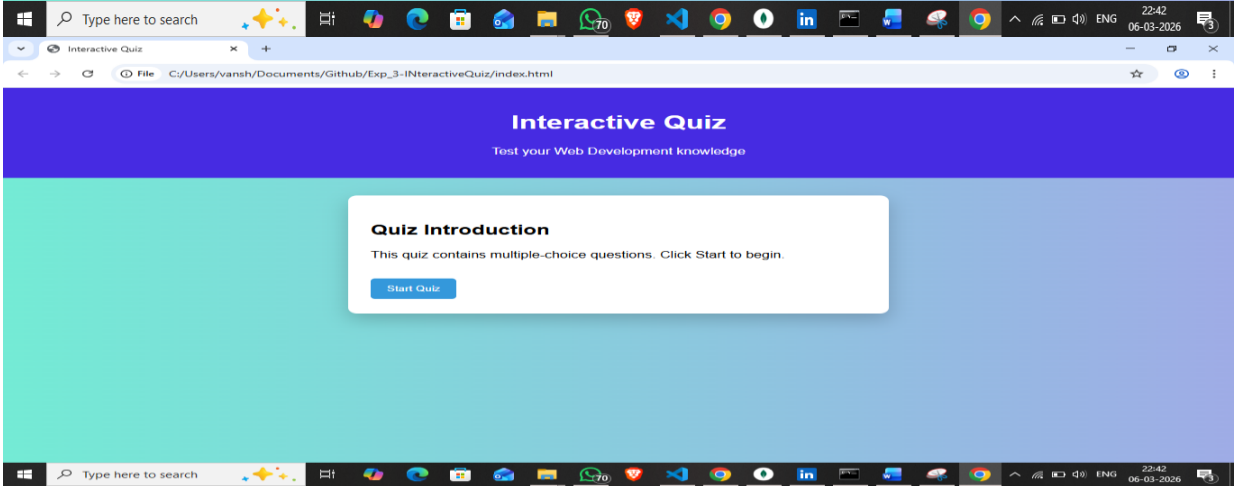
```
function startQuiz() {
    document.getElementById("intro").classList.add("hidden");
    document.getElementById("quiz").classList.remove("hidden");
    loadQuestion();
}

function loadQuestion() {
    const q = quizData[currentQuestion];
    document.getElementById("question").innerText = q.question;

    const optionsDiv = document.getElementById("options");
    optionsDiv.innerHTML = "";

    q.options.forEach((option, index) => {
        const div = document.createElement("div");
        div.className = "option";
        div.innerText = option;
        div.onclick = () => selectAnswer(index);
        optionsDiv.appendChild(div);
    });
}

function selectAnswer(selected) {
    if (selected === quizData[currentQuestion].correct) {
        score++;
    }
    nextQuestion();
}
```

```
function nextQuestion() {
    currentQuestion++;
    if (currentQuestion < quizData.length) {
        loadQuestion();
    } else {
        showResult();
    }
}
function showResult() {
    document.getElementById("quiz").classList.add("hidden");
    document.getElementById("result").classList.remove("hidden");

    document.getElementById("scoreText").innerText =
        `Your Score: ${score} / ${quizData.length}`;
    let feedback = "Try Again";
    if (score === quizData.length) feedback = "Excellent";
    else if (score >= 2) feedback = "Good";
    document.getElementById("feedback").innerText = feedback;
}
function restartQuiz() {
    currentQuestion = 0;
    score = 0;
    document.getElementById("result").classList.add("hidden");
    document.getElementById("intro").classList.remove("hidden")
}
```

> Output :


Interactive Quiz
C:/Users/vansh/Documents/Github/Exp_3-INteractiveQuiz/index.html
Interactive Quiz
Test your Web Development knowledge
Which language is used for styling web pages?
HTML
CSS
Java
Python
Next


Interactive Quiz
C:/Users/vansh/Documents/Github/Exp_3-INteractiveQuiz/index.html
Interactive Quiz
Test your Web Development knowledge
What does HTML stand for?
Hyper Text Markup Language
High Text Machine Language
Hyperlinks Text Mark Language
None of these
Next


Interactive Quiz
C:/Users/vansh/Documents/Github/Exp_3-INteractiveQuiz/index.html
Interactive Quiz
Test your Web Development knowledge
Quiz Introduction
This quiz contains multiple-choice questions. Click Start to begin.
Start Quiz

Experiment : 4

❖ **Aim :**
To design and develop a **Dynamic To-Do List Web Application** using **HTML, CSS, and JavaScript** that allows users to add, edit, delete, and manage tasks interactively.

❖ **Tasks :**

• **Project Setup & Introduction** – Create a project folder named **dynamic_todo_list** and add linked HTML, CSS, and JavaScript files with a short description of the application.

• **To-Do List Interface Design** – Create an input field to enter tasks, an **Add Task** button, and display tasks in a list format using HTML.

• **Styling with CSS** – Apply CSS styling to input fields, buttons, and task list to ensure proper alignment, spacing, and visual clarity.

• **Add & Display Tasks Using JavaScript** – Capture user input, add tasks dynamically to the list, and prevent empty tasks from being added.

• **Edit & Delete Task Features** – Provide **Edit** and **Delete** buttons for each task to allow modification and removal of tasks dynamically.

• **Task Completion Status (Bonus)** – Add a checkbox or toggle to mark tasks as completed and visually indicate completed tasks using a strikethrough effect.

❖ **Tech Stack Used :**
• **HTML** • **CSS** • **JavaScript**

❖ Html Code :

```
<!--

Project Title: Dynamic To-Do List

Name: Vansh

Experiment: 4

Date: 2026

-->

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">
```

```html
    <title>Dynamic To-Do List</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<header>
    <h1>My To-Do List</h1>
    <p>A simple to-do list to add, edit, delete, and complete tasks</p>
</header>
<main class="container">
    <div class="input-area">
        <input type="text" id="taskInput" placeholder="Enter a new task">
        <button onclick="addTask()">Add Task</button>
    </div>

    <ul id="taskList">
    </ul>

</main>

<footer>
    <p>© 2026 Vansh </p>
</footer>

<script src="script.js"></script>
</body>
</html>
```

❖ CSS Code :

```
* {
    box-sizing: border-box;
    font-family: Arial, sans-serif;
}
body {
    margin: 0;
    background-color: white;
}
header {
    background-color: blueviolet;
    color: white;
    padding: 20px;
    text-align: center;
}
.container {
    background-color: white;
    max-width: 500px;
    margin: 30px auto;
    padding: 20px;
    border-radius: 10px;
}

.input-area {
    display: flex;
    gap: 10px;
}
```

```
input {
    flex: 1;
    padding: 10px;
    font-size: 14px;
}

button {
    padding: 10px 15px;
    background-color: blue;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background-color: #3b55a0;
}

ul {
    list-style: none;
    padding: 0;
    margin-top: 20px;
}

li {
    background-color: whitesmoke;
    padding: 10px;
```

```
    margin-bottom: 10px;
    border-radius: 6px;
    display: flex;
    align-items: center;
    justify-content: space-between;
}

.completed {
    text-decoration: line-through;
    color: gray;
}

.task-buttons button {
    margin-left: 5px;
    font-size: 12px;
}

footer {
    text-align: center;
    padding: 10px;
    color: black;
}
```

❖ Javascript Code :

```
function addTask() {
    const input = document.getElementById("taskInput");
    const taskText = input.value.trim();
```

```
if (taskText === "") {
    alert("Please enter a task");
    return;
}

const li = document.createElement("li");

const checkbox = document.createElement("input");
checkbox.type = "checkbox";
checkbox.onclick = () => toggleTask(li);

const span = document.createElement("span");
span.innerText = taskText;

const btnDiv = document.createElement("div");
btnDiv.className = "task-buttons";

const editBtn = document.createElement("button");
editBtn.innerText = "Edit";
editBtn.onclick = () => editTask(span);

const deleteBtn = document.createElement("button");
deleteBtn.innerText = "Delete";
deleteBtn.onclick = () => deleteTask(li);

btnDiv.appendChild(editBtn);
btnDiv.appendChild(deleteBtn);
```

```
    li.appendChild(checkbox);
    li.appendChild(span);
    li.appendChild(btnDiv);

    document.getElementById("taskList").appendChild(li);

    input.value = "";
}

function deleteTask(task) {
    task.remove();
}

function editTask(span) {
    const newText = prompt("Edit your task:", span.innerText);
    if (newText !== null && newText.trim() !== "") {
        span.innerText = newText;
    }
}

function toggleTask(task) {
    task.classList.toggle("completed");
}
```

❖ Server:

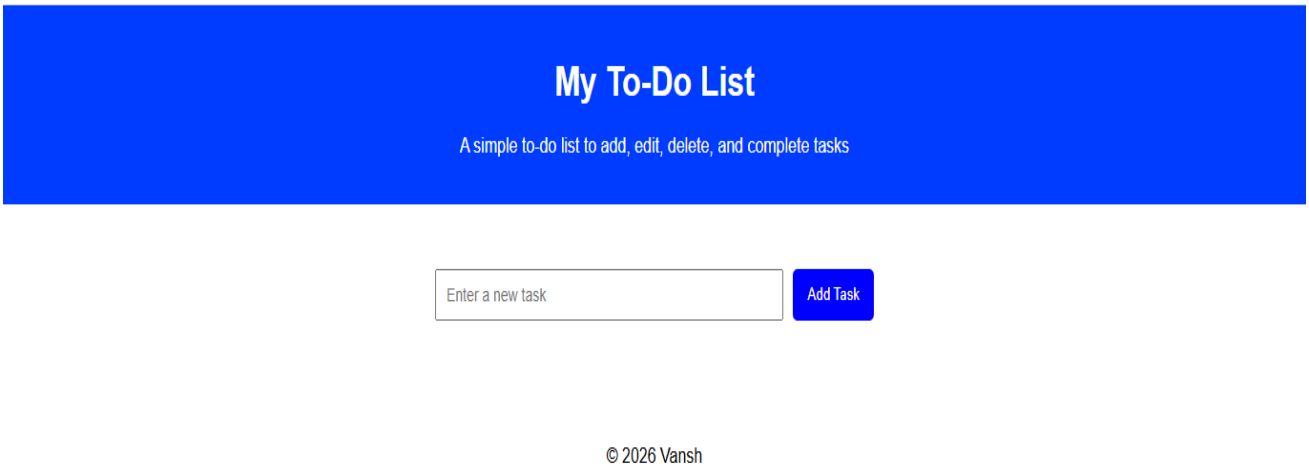
```
require('dotenv').config();
const express = require('express');
```

```
const cors = require('cors');
const nodemailer = require('nodemailer');

const app = express();
app.use(cors());
app.use(express.json());

const SMTP_HOST = process.env.SMTP_HOST;
const SMTP_PORT = process.env.SMTP_PORT || 587;
const SMTP_USER = process.env.SMTP_USER;
const SMTP_PASS = process.env.SMTP_PASS;
const RECIPIENT_EMAIL = process.env.RECIPIENT_EMAIL;
const REMINDER_MINUTES = parseInt(process.env.REMINDER_MINUTES || '15', 10);

if (!SMTP_HOST || !SMTP_USER || !SMTP_PASS || !RECIPIENT_EMAIL) {
  console.warn('SMTP credentials or RECIPIENT_EMAIL not fully configured.
See .env.example');
}

let transporter;
async function createTransporter() {
  if (SMTP_HOST && SMTP_USER && SMTP_PASS) {
    return nodemailer.createTransport({
      host: SMTP_HOST,
      port: Number(SMTP_PORT),
      secure: Number(SMTP_PORT) === 465,
      auth: { user: SMTP_USER, pass: SMTP_PASS }
    });
```

```
  }

  // Fallback: create a test account (Ethereal) for development/testing
  const testAccount = await nodemailer.createTestAccount();
  console.log('Using Nodemailer test account. Messages will be available at preview URLs.');
  return nodemailer.createTransport({
    host: 'smtp.ethereal.email',
    port: 587,
    auth: { user: testAccount.user, pass: testAccount.pass }
  });
}

(async () => { transporter = await createTransporter(); })();

// Simple in-memory store of scheduled timers (not persisted)
const scheduled = new Map();

app.get('/', (req, res) => {
  res.send('To-Do Reminder Server running');
});

// POST /api/tasks { text, due }
app.post('/api/tasks', (req, res) => {
const { text, due } = req.body;

  if (!text) return res.status(400).json({ error: 'Task text required' });
```

```
if (!due) {
  // If no due time provided, send immediate notification
  sendReminderEmail(text, new Date())
    .then(() => res.json({ status: 'sent_immediate' }))
    .catch(err => res.status(500).json({ error: err.message }));
  return;
}

const dueDate = new Date(due);
if (isNaN(dueDate)) return res.status(400).json({ error: 'Invalid due date' });

const reminderTime = new Date(dueDate.getTime() - REMINDER_MINUTES * 60000);
const delay = reminderTime.getTime() - Date.now();

const id = Date.now() + Math.random().toString(36).slice(2, 8);

if (delay <= 0) {
  // reminder time passed — send immediately
  sendReminderEmail(text, dueDate)
    .then(() => res.json({ status: 'sent_immediate' }))
    .catch(err => res.status(500).json({ error: err.message }));
  return;
}

const timer = setTimeout(() => {
  sendReminderEmail(text, dueDate)
    .catch(err => console.error('Failed to send reminder:', err.message));
  scheduled.delete(id);
```

```
  }, delay);

  scheduled.set(id, { text, due: dueDate.toISOString(), timer });

  res.json({ status: 'scheduled', id, sendAt: reminderTime.toISOString() });
});

async function sendReminderEmail(taskText, dueDate) {
  if (!transporter || !RECIPIENT_EMAIL) return Promise.reject(new Error('Email not configured'));

  const formattedDue = dueDate ? new Date(dueDate).toLocaleString() : 'soon';
  const mailOptions = {
    from: SMTP_USER || 'no-reply@example.com',
    to: RECIPIENT_EMAIL,
    subject: `Reminder: upcoming task — ${taskText}`,
    text: `Reminder: Your task "${taskText}" is due at ${formattedDue}. This is an automated reminder sent ${REMINDER_MINUTES} minutes before.`
  };

  try {
    const info = await transporter.sendMail(mailOptions);
    // Log preview URL for test accounts (Ethereal)
    const preview = nodemailer.getTestMessageUrl(info);
    if (preview) console.log('Preview URL:', preview);
    return info;
  } catch (err) {
    return Promise.reject(err);
```

```
  }
}

// end of file

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Reminder server listening on port ${PORT}`);
});
```

Output :